

1 Planning 策略——ReAct + Plan&Execute 混合模式

LeisureAgent 由 18 个 LangGraph 节点按 ReAct（推理-行动-观察）与 Plan&Execute（计划-执行）混合模式协作运行。LLM 调用失败时自动降级规则逻辑，确保核心流程不中断。

1.1 工作流与路由

```
load_session → classify_intent [路由]
casual / out_of_domain → direct_reply → END
inquiry → search_inquiry → present_inquiry → END
new_plan → analyze_goal → search_candidates → detect_exceptions
    critical_gap & retry<2 → adjust_search → search_candidates (loop)
    → compose_plan → persist_plan
    auto_execute → execute_bookings → finalize_executed → END
    manual → present_plan → END
feedback → analyze_feedback → (search?) → compose → ... → END
confirm → execute_bookings
    failure & retry<2 → replan_execute → persist → execute (loop)
    ok → finalize_executed → END
```

意图分类采用三层优先级：

- (1) **快速规则**：确认/反馈关键词 + pending 方案 → 直接路由；工作日关键词 → rejection。命中即返回，不调 LLM；
- (2) **LLM 分类**：轻量模型（temperature=0, max_tokens=512），一站式输出意图类型 + 直接回复文案；
- (3) **TF-IDF 语义降级**：char n-gram + 余弦相似度，支持模糊匹配（“哈喽”→CASUAL、“推荐好吃的”→INQUIRY）。
场景驱动编排：三种出行场景各有独立的候选优先级与编排参数：

场景	活动候选优先级	餐厅策略	特征
family	amusement_park scenic_spot	> 偏好火锅/中餐	停留 100 min，儿童友好
friends	exhibition_hall scenic_spot	> 默认第一项	停留 90 min，拍照聊天
other	amusement > scenic exhibition	> 默认第一项	停留 90 min，通用

表 1：场景配置

LLM 生成方案后经**四层保障**确保质量：JSON 严格校验（提取 → 清洗 → Pydantic 校验，最多 3 次重试）；多日校验（day_num 分布不符 → LLM 重试 → 规则强制拆分）；结构安全网（缺失 dining/play → 自动补齐）；LLM 降级（调用失败 → 规则编排兜底）。

编排硬约束：活动时序固定（游玩 → 缓冲 → 用餐）；用餐不重复（用户指定优先）；停留 5-360 min；多日每天至少 1 项。

2 工具调用链路

2.1 四大核心链路

2.2 咨询模式与并发安全

咨询模式 (inquiry)：用户输入含领域关键词但非规划意图时，走轻量级浏览路径：search_inquiry_node 搜索五类场所 → present_inquiry_node 返回 InquiryEvent（含可用性、距离、排队时间）→ 前端展示 Yes/No/Other 弹窗。

并发安全：SQLite 单连接 + threading.Lock() 写锁 + WAL 模式（并发读、串行写）。预约操作使用 BEGIN IMMEDIATE + UPDATE WHERE current < max 原子 SQL，rowcount=0 判满，杜绝 TOCTOU 竞态。

3 异常处理机制

React (HashRouter + SSE) → FastAPI (/api/*) → LangGraph (18 nodes) → Tools (search/booking/location) → Service/Re

链路	输入	关键步骤	输出
搜索 (search)	constraints (场景/距离/菜系)	遍历 restaurant / scenic_spot / amusement_park / exhibition_hall / mall 五类表筛选 _extract_named_locations() 强制纳入用户提及地点 标记 available 字段	candidates: dict[category, list[item]]
异常检测 (detect)	candidates (全部候选)	_check_availability() 检查容量 判断 critical_gaps → 触发搜索自愈 (≤2 次) 返回 exceptions + warnings	exceptions + warnings
持久化 (persist)	AgentPlan (含 3-8 项)	创建 travel_plan 主记录 for each item: 场馆存在 → 名额 → 时段冲突检测 (仅同 day_num 内) → INSERT 失败项保留在内存方案, WARNING 日志	persisted plan (DB + 内存)
执行预约 (execute)	plan_id	preflight_check() 预检查 无需预约 → skip; 无容量 → 直接标记 有容量 → BEGIN IMMEDIATE + 原子 UPDATE WHERE current < max rowcount=0 → 已满; 失败 → 同类替代重试 (≤2 次)	booking_ results

表 2: 核心工具调用链路

机制	覆盖节点	处理策略
LLM 降级	classify / analyze / compose	LLM 调用失败 → 规则逻辑兜底: TF-IDF 语义匹配替代 LLM 分类 (classify)、默认约束提取 (analyze)、场景配置驱动编排 (compose)。关键路径不依赖 LLM 可用性。
ReAct 搜索自愈	detect → adjust → search	关键类别无可用候选 → 放宽距离约束 50% → 重新搜索。上限 2 次, 耗尽后通过 gap_report 告知用户缺口, 不给出发合方案。
ReAct 执行自愈	execute → replan → persist → execute	预约失败 → 从原始 candidates 找同类替代地点 (不调 LLM, 纯规则匹配) → 重新持久化并执行。上限 2 次。
预约名额不足	persist / execute	持久化时检测 current_booking_count ≥ max_booking_count → 返回”预约名额已满”错误; 执行时原子 UPDATE WHERE current < max, rowcount=0 判定名额已满, 触发执行自愈寻找替代。
结构安全网	compose	LLM 产出缺少 dining (用餐) 或 play (游玩) 项 → 从候选池按场景优先级确定性补齐, 每次触发记录 safety_net 指标用于追踪 LLM 产出质量。
输入安全	load_session	22 个正则模式覆盖中英文: 指令覆盖、角色劫持、提示词提取、SQL 注入标记。控制字符与零宽字符自动清洗。2000 字符上限。拦截后返回 guard_reject 事件。
结构化输出保护	invoke_structured	JSON 提取 → 语法清洗 (单引号/尾部逗号/中文标点) → Pydantic 校验 → 业务校验 → 错误回传 LLM 重试。最多 3 次, 3 次均失败则抛异常触发上层降级。
配置校验	启动时	validate_config() 检查当前 provider 对应的 API key 及必需配置, 缺失则立即 WARNING, 不等到首次 LLM 调用才报错。
可观测性	全局	JSON 结构化日志输出: LLM 调用追踪 (节点名、耗时、成功/失败、token 用量); 安全网触发计数。数据持久化到 SQLite metrics_counter 表, GET /api/agent/metrics 端点可查询。

表 3: 九层异常处理机制